

关于AS3的事件移除释疑

as3.0中的事件Event（位于包flash.events内，继承至Object，子类有...）

包

flash.events

类

public class Event

继承

Event → Object

子类

ActivityEvent, CaptionChangeEvent, CaptionTargetEvent, ColorPickerEvent, ComponentEvent, ContextMenuEvent, DataChangeEvent, FocusEvent, HTTPStatusEvent, KeyboardEvent, LayoutEvent, ListEvent, MetadataEvent, MotionEvent, MouseEvent, NetStatusEvent, ProgressEvent, ScrollEvent, SliderEvent, SoundEvent, StatusEvent, SyncEvent, TextEvent, TimerEvent, TweenEvent, VideoEvent

语言版本

: ActionScript 3.0

Player 版本

: Flash Player 9

既然是释疑，主要是在与同事聊天的过程中提及的几个问题：

- 1、addEventListener重复监听是否会被多次调用？
- 2、如何移除一个匿名函数的监听？
- 3、target、currentTarget之间的区别
- 4、关于Event对象的其它需要注意的地方

/*****

* addEventListener重复监听是否会被多次调用？

*****/

这里重点在于“重复”两个字，如果监听一个对象，事件类型相同，监听函数也是相同，则该监听函数在事件被触发时只被调用一次。如果监听的事件类型不同或是监听函数不同，则该监听函数，在该事件类型被触发时调用。

```
//重复的监听只会被调用一次
mc.addEventListener(MouseEvent.CLICK, onClickHandler);
mc.addEventListener(MouseEvent.CLICK, onClickHandler);
mc.addEventListener(MouseEvent.CLICK, onClickHandler);
```

上面的写法，onClickHandler函数在对象的CLICK事件被触发时被调用一次。

```
//移除指定的监听事件
mc.removeEventListener(MouseEvent.CLICK, onClickHandler);
```

使用removeEventListener则会移除该事件，CLICK事件被触发时不再调用onClickHandler函数。

如果监听函数为匿名函数：

```
//使用匿名函数进行事件的监听
mc.addEventListener(MouseEvent.CLICK, function(e:MouseEvent):void {
    var _mc:MovieClip = e.target as MovieClip;
    _mc.removeEventListener(MouseEvent.CLICK, arguments.callee);

    setTxt('~haha~我就是匿名函数【remove】');
});
```

需要移除匿名函数时，一是可以使用变量保存该匿名函数的引用，二是可以在该匿名函数内使用arguments.callee获取该匿名函数的引用，使用removeEventListener移除对指定事件类型的监听。

使用闭包获得的匿名函数，每次调用时获得的匿名函数都不是同一个引用，所以对事件的监听将会被多次调用。例如有一个函数为：getClickFn()。

```
private function getClickFn(n:Number=0, fn:Function=null):Function
{
    var num:Number = n;

    return function(evt:MouseEvent):void {
        fn == fn(arguments.callee, evt);

        setTxt('当前num的值: ' + (++num));
    }
}
```

当多次使用该函数进行监听时，将会被多次调用。

```
mc.addEventListener(MouseEvent.CLICK, getClickFn(50));
```

而addEventListener方法，属于类EventDispatcher(flash.events，继承至Object，实现IEventDispatcher)，子类有(DisplayObject...)

包

flash.events

Class

public class EventDispatcher

继承

EventDispatcher → Object

Implements

IEventDispatcher

Subclasses

AbstractInvoker, AsyncToken, AxisBase, Camera, Channel, ChannelSet, ChartItem, ConstraintColumn, ConstraintRow, CSSStyleDeclaration, DataTransform, DisplayObject, Effect, EffectInstance, EffectManager, FileReference, FileReferenceList, FileStream, FlexClient, FlexNativeMenu, GradientBase, GradientEntry, HierarchicalCollectionView, HierarchicalCollectionViewCursor, HierarchicalData, Icon, IME, LayoutManager, LoaderInfo, LocalConnection, LogLogger, MessageAgent, Microphone, ModuleBase, NativeApplication, NativeMenu, NativeMenuItem, NativeWindow, NetConnection, NetStream, PrintJob, RadioButtonGroup, ResourceManagerImpl, Screen, ServiceMonitor, SetEventHandler, SharedObject, Socket, SolidColor, Sort, SortField, Sound, SoundChannel, SQLConnection, SQLStatement, State, StreamingConnectionHandler, Stroke, StyleSheet, Timer, ToolTipManager, Tween, UIComponentAutomationImpl, UIMovieClipAutomationImpl, URLLoader, URLStream, Validator, XMLSignatureValidator, XMLSocket

语言版本

: ActionScript 3.0

RuntimeVersions:

AIR 1.0, Flash Player 9

该方法有5个参数。addEventListener(eventType, listenerFn, useCapture:Boolean=false, priority:int=0, useWeakReference:Boolean=false);

一般很少用到后面的参数，分别是：

- useCapture -> 是否在捕获阶段监听事件，默认在冒泡阶段进行监听
- priority -> 默认为0，也就是监听事件时，回调监听函数的“层次”顺序，比如之后监听的函数要在事件触发时有拦截操作等（取消事件冒泡，阻止其后事件的触发stopPropagation、stopImmediatePropagation）
- useWeakReference -> 是否为弱引用，强烈建议避免使用该参数，不易控制。

hasEventListener用来检查对象是否为特定的事件类型注册了监听器。

* **target**、**currentTarget**的区别

*****/

这个是比较容易混淆，但也非常常见。一句话概括来讲就是：

target -- 触发事件的事件源，**currentTarget** -- 触发事件的监听对象。

/*****

* 关于**Event**其它要注意的地方

*****/

首先要很清楚，一个事件它经历了哪几个阶段：捕获阶段 -> 目标阶段 -> 冒泡阶段



此内容无法在框架中显示

为了帮助保护在此网站中输入的信息的安全，此内容的发行者不允许在框架中显示该信息。

然后就是**stopPropagation**、**stopImmediatePropagation**这两个方法它们的区别，写个例子就知道了<从方法的说明中就能知道大概意思，写个**Demo**加深印象就可以了>

写了一堆，最后还是写一个**demo**，比较实际一点：

```
1: package
2: {
3:     import flash.display.MovieClip;
4:     import flash.display.Sprite;
5:     import flash.events.MouseEvent;
6:     import flash.filters.GlowFilter;
7:     import flash.text.TextField;
8:     import flash.text.TextFieldAutoSize;
9:     import flash.text.TextFieldType;
10:    import flash.text.TextFormat;
11:    /**
12:     * ...
13:     * @author Meteoric
14:     */
15:    public class EventDemo extends Sprite
16:    {
17:
18:        public function EventDemo()
19:        {
20:            initView();
21:        }
22:
23:        private var txt:TextField;
24:
```

```
25:         private function getTextGlowFilter(colorVal:uint=0x000000, blur:Number=2):GlowFilter
26:         {
27:             var color:uint = colorVal;
28:             var alpha:Number = 1;
29:             var blurX:Number = blur;
30:             var blurY:Number = blur;
31:             var strength:Number = 255;
32:             var quality:Number = 1;
33:
34:             return new GlowFilter(color, alpha, blurX, blurY, strength, quality);
35:         }
36:
37:     private function initView():void
38:     {
39:         txt = new TextField();
40:         txt.x = 200;
41:         txt.y = 50;
42:         txt.selectable = false;
43:         txt.mouseEnabled = false;
44:         txt.autoSize = TextFieldAutoSize.LEFT;
45:         txt.width = 350;
46:         //txt.height = 100;
47:         txt.wordWrap = true;
48:         txt.multiline = true;
49:         txt.type = TextFieldType.INPUT;
50:         txt.background = true;
51:         txt.backgroundColor = 0x232323;
52:
53:         var txtfor:TextFormat = new TextFormat();
54:         txtfor.size = 15;
55:         txtfor.font = "verdana";
56:         txtfor.bold = true;
57:         txtfor.color = 0xffffffff;
58:
59:         txt.defaultTextFormat = txtfor;
60:         txt.filters = [getTextGlowFilter(0xffcc66)];
61:
62:         addChild(txt);
63:
64:         //监听事件
65:         var mc:MovieClip = new MovieClip();
66:
67:         mc.graphics.clear();
68:         mc.graphics.beginFill(0xcc0000, .7);
69:         mc.graphics.drawCircle(100, 100, 50);
70:         mc.graphics.endFill();
71:
72:         addChild(mc);
73:
74:         mc.buttonMode = true;
75:
76:
77:         //重复的监听只会被调用一次
78:         mc.addEventListener(MouseEvent.CLICK, onClickHandler);
79:         mc.addEventListener(MouseEvent.CLICK, onClickHandler);
80:         mc.addEventListener(MouseEvent.CLICK, onClickHandler);
```

```
81:
82:         //检测是否绑定了指定的事件
83:         setTxt( '是否监听了MouseEvent.CLICK事件: ' + mc.hasEventListener(MouseEvent.CLICK));
84:
85:         //移除指定的监听事件
86:         mc.removeEventListener(MouseEvent.CLICK, onClickHandler);
87:
88:         //重复监听的事件，移除一次后将不再被监听
89:         setTxt( '是否监听了MouseEvent.CLICK事件: ' + mc.hasEventListener(MouseEvent.CLICK));
90:
91:         //使用匿名函数进行事件的监听
92:         mc.addEventListener(MouseEvent.CLICK, function(e:MouseEvent):void {
93:             var _mc:MovieClip = e.target as MovieClip;
94:             _mc.removeEventListener(MouseEvent.CLICK, arguments.callee);
95:
96:             setTxt( '~~haha~~我就是匿名函数【remove】 ');
97:         });
98:
99:         //下面使用getClickFn获得的是两个不同的匿名函数引用
100:        mc.addEventListener(MouseEvent.CLICK, getClickFn(100, function(fn:Function, evt:MouseEvent):void {
101:
102:            var _mc:MovieClip = evt.target as MovieClip;
103:
104:            _mc.removeEventListener(MouseEvent.CLICK, fn);
105:
106:            setTxt( "~~hehe~~另外一个匿名函数【remove】 ");
107:
108:        }), false, 1);
109:
110:        mc.addEventListener(MouseEvent.CLICK, getClickFn(50));
111:    }
112:
113:    private function onClickHandler(e:MouseEvent):void
114:    {
115:        setTxt("----onClickHandler" + getNow());
116:    }
117:
118:    private function getClickFn(n:Number=0, fn:Function=null):Function
119:    {
120:        var num:Number = n;
121:
122:        return function(evt:MouseEvent):void {
123:            fn && fn(arguments.callee, evt);
124:
125:            setTxt( '当前num的值: ' + (++num));
126:        }
127:    }
128:
129:    private function getNow():String
130:    {
131:        var date:Date = new Date();
132:        var year:Number = date.getFullYear();
133:        var month:Number = date.getMonth() + 1;
134:        var day:Number = date.getDay();
135:
136:        var hours:Number = date.getHours();
```

```
137:         var minutes:Number = date.getMinutes();
138:         var seconds:Number = date.getSeconds();
139:
140:         return [year, month, day].join("-") + " " + [hours, minutes, seconds].join(":");
141:     }
142:
143:     private function setTxt(str:String):void
144:     {
145:         txt.appendText(str + '\n');
146:
147:         trace(str);
148:     }
149:
150: }
151:
152: }
```

分类: [Actionscript, Flex/Flash/AS3](#)

好文要顶

关注我

收藏该文

 meteoric_cry

关注 - 10

粉丝 - 211

[+加关注](#)

[« 上一篇: JavaScript的Hook](#)

1

推荐

0

反对